

Finetune LLMs.

① Quantization

Reduce the size of the model
Comparison Technique that maps

high precision value to lower precision values
higher Memory formats to lower Memory formats.

modes of Quantization

- ③ Post training Quantization (PQT)
- ④ Quantization Aware Training (QAT)

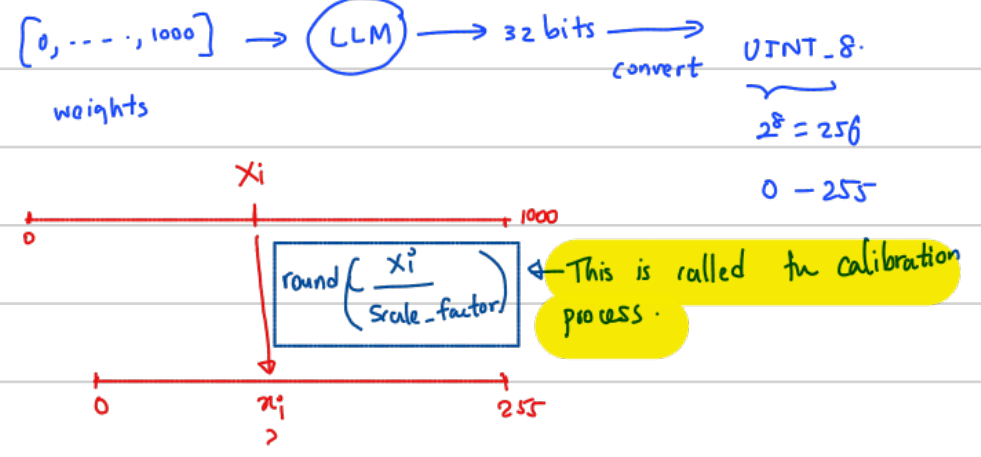
PQT - This is done After LLM Trained.
QAT - much better / No need of Quantization / Easy

Perform Quantization

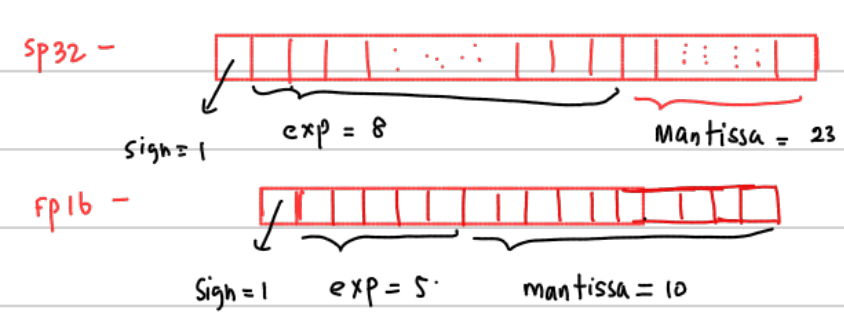
① Symmetric Quantization.

Batch Normalization - Entire distribution of the weights are centered nearly on zero

② Symmetric UINT8 Quantization.



$$\text{scale_factor} = \frac{X_{\max} - X_{\min}}{Q_{\max} - Q_{\min}}$$



② Asymmetric UINT8 Quantization.

④ Not Symmetrically distributed.

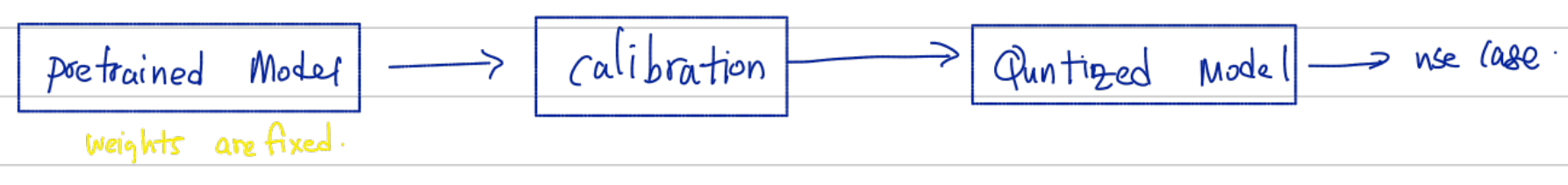
$[-20, \dots, 1000]$ s.f = $255-1$
 $[0, \dots, 255]$

$$\text{s.f} = \frac{1000 + 20}{255} = +4$$

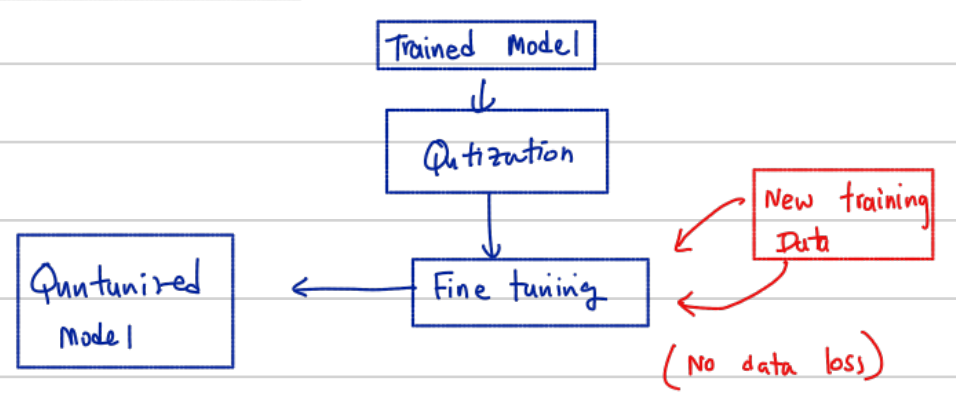
$$\text{round}(-20/4) = -5 + 5 = 0$$

zero point at = +5//
every time we add +5//

Post training Quantization -> can be loss of data.



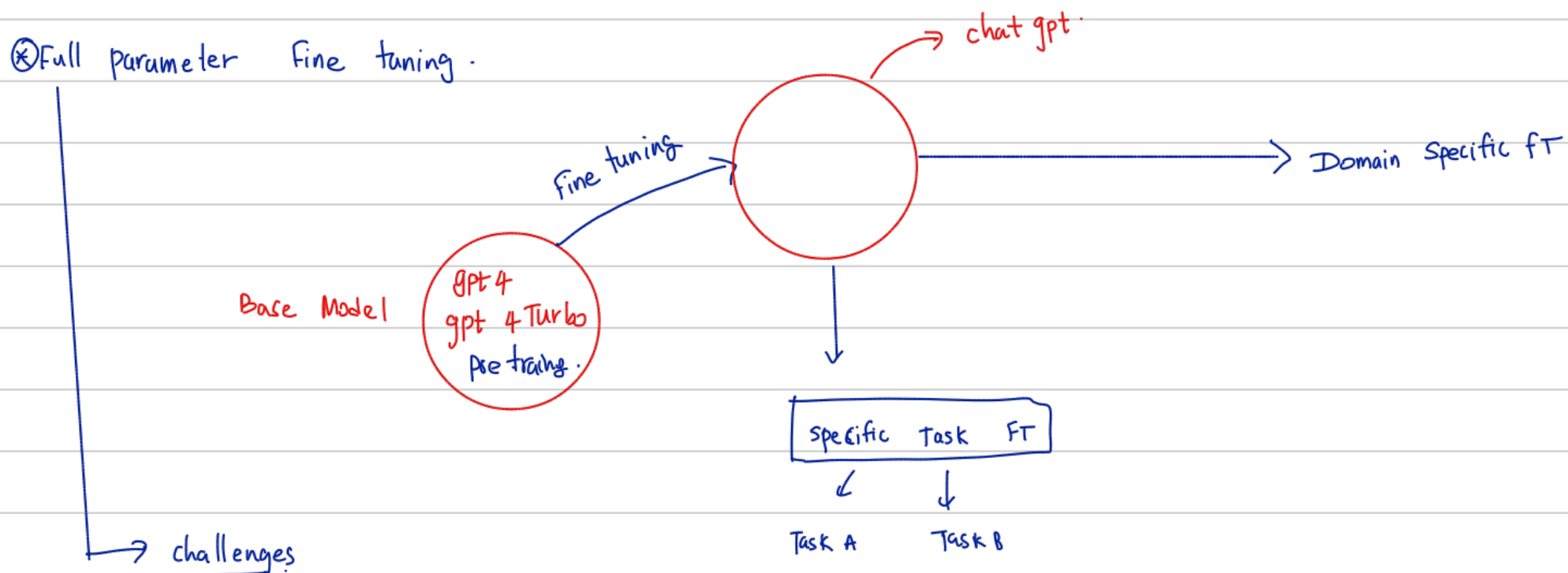
Quantization Aware training



LORA & QLORA

LORA → Low rank Adaptation of LLM's

QLORA → Quantized Low rank Adaptation of LLM's



① update all model weights (175B param)

② Hardware Resource constraint (Downstream tasks / GPU, RAM)

What LORA Does?

Instead of updating all parameters it will track changes

pre trained weights

LORA Tracked weights

Fine tuned weights



$$W_D + \Delta W = W_D + BA$$

Set the Rank

Matrix decomposition

$$B \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}_{3 \times 1} \times \begin{bmatrix} 4 & 5 & 6 \end{bmatrix}_{1 \times 3} =$$

Rank = 1

Rank ↑ efficient ↑

" " param ↑ (Trainable).

High Rank use for when model wants to learn complex things

QLORA ⇒ Quantized - LORA

F16 ⇒ 4 bit.

Not use much memory

The Era of 1-bit LLM

BitNet b1.58 weights are in $\{-1, 0, 1\}$

LLama fp32

Quantization

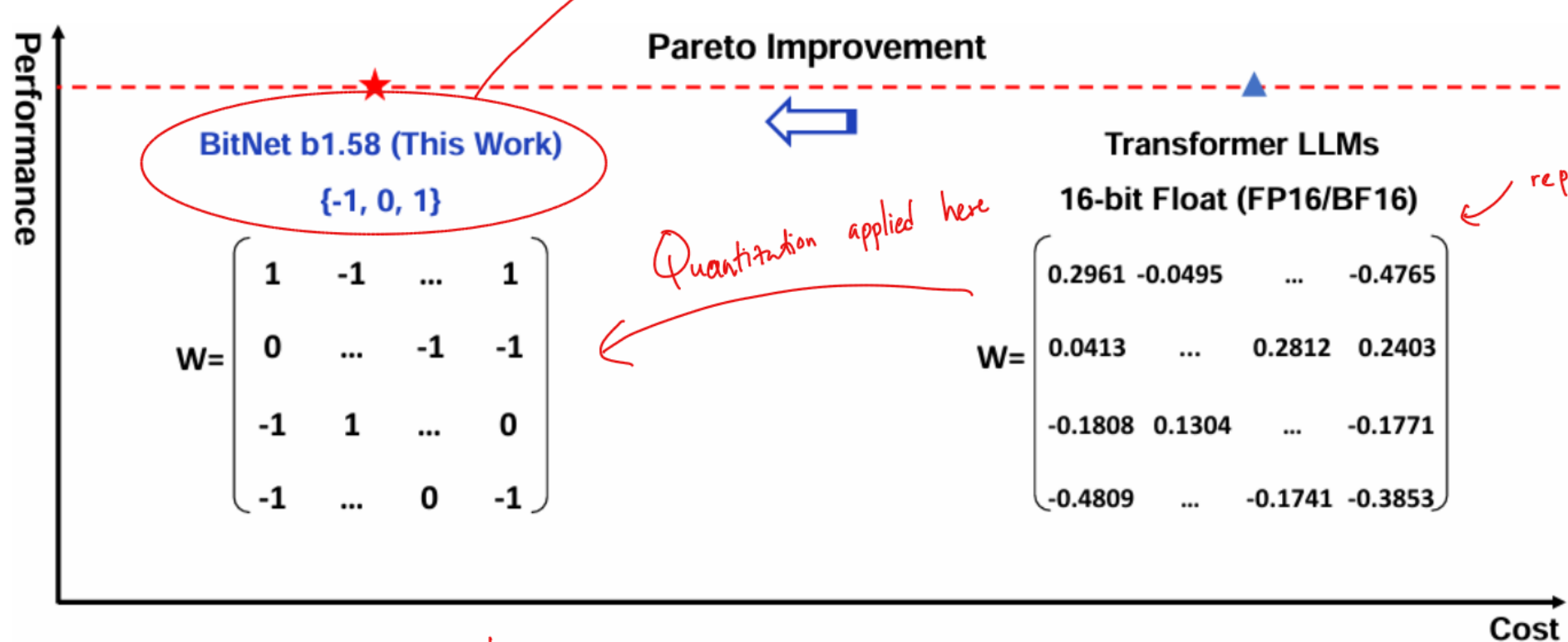
using Lora & Qlora Fintune.

can convert into 1 bit

In the training (1.58 bit weights & 8 bit Activations)

Based on BitNet Architecture (Transformer replaces unLinear with Bit Linear)

All LLM are in 1.58 bits



$\sum w_m \times b$

+ bias

Cost

$Y = f(W, X)$

Model W

$\times$

Input X

$=$

Output Y

FP16

$$\begin{bmatrix} 0.2961 & -0.0495 & -0.0924 & -0.4765 \\ 0.0413 & 0.3397 & 0.2812 & 0.2403 \\ -0.1808 & 0.1304 & 0.4322 & -0.1771 \\ -0.4809 & 0.3244 & -0.1741 & -0.3853 \end{bmatrix}$$

$\times$

$$\begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$=$

$$\begin{bmatrix} 0.2961x_0 - 0.0495x_1 - 0.0924x_2 - 0.4765x_3 \\ \dots \\ x_0 - x_1 - x_2 + x_3 \end{bmatrix}$$

Multiplication-then-Addition

GPU

$\downarrow$

only integer addition

New Hardware

Only addition.
 * Gpu get reduced.

1(.58)-bit

$$\begin{bmatrix} 1 & -1 & -1 & 1 \\ 0 & 1 & -1 & -1 \\ -1 & 0 & 1 & -1 \\ -1 & 1 & 1 & 0 \end{bmatrix}$$

$\times$

$$\begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$=$

$$\begin{bmatrix} x_0 - x_1 - x_2 + x_3 \\ \dots \end{bmatrix}$$

Addition (no Multiplication)

* forward propagation

Quantization Function

Absmean Quantization Function

1) scales the weight matrix by its avg absolute value then round to the nearest Integer

$$\tilde{w} = \text{RoundClip}\left(\frac{w}{\sigma + \epsilon}, -1, 1\right)$$

$$\text{RoundClip}(\sigma, a, b) = \max(a, \min(b, \text{round}(\sigma)))$$

$$\sigma = \frac{1}{nm} \sum_{ij} |w_{ij}|$$